Assignment1 - Credit Card Analysis

Import Libraries/Dataset

1. Import the required libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
```

2. Download the dataset

```
!wget -q https://bit.ly/3kXTdox
```

Data Visualization and Exploration

3. Read/ingest/load the dataset into a dataframe. Name the features present in the dataset and if the target variable is present or not.

```
df = pd.read_excel('default of credit card clients.xls')
```

```
df.shape
```

```
(30000, 25)
```

```
df.columns
```

```
Index(['ID', 'LIMIT_BAL', 'SEX', 'EDUCATION', 'MARRIAGE', 'AGE', 'PAY_0',
       'PAY_2', 'PAY_3', 'PAY_4', 'PAY_5', 'PAY_6', 'BILL_AMT1',
       'BILL_AMT2', 'BILL_AMT3', 'BILL_AMT4', 'BILL_AMT5', 'BILL_AMT6',
       'PAY_AMT1', 'PAY_AMT2', 'PAY_AMT3', 'PAY_AMT4', 'PAY_AMT5',
       'PAY_AMT6', 'default payment next month'],
      dtype=object)
```

Yes the target variable is present. The last column the dataset is the target variable which is default payment next month.

4. Comment on class imbalance with appropriate visualization method

```
plt.pie(df['default payment next month'].value_counts(),
        labels=['Not Default', 'Default'],
        autopct='%1.1f%%')
plt.legend(df['default payment next month'].value_counts().index.tolist())
plt.title('Default Payment Next Month')
```

As the pie chart shows it possesses, the target variable dataset is not well balanced with the class imbalance ratio of 78:22 in favour of Not Default.

5. Use appropriate visualizations to get an insight about the dataset.

```
df['SEX'].value_counts()
```

```
2    18112
1    11888
Name: SEX, dtype: int64
```

```
df['SEX'].hist(bins=2)
```

Less than half of the Total transactions were in Gender 1 and more than 60% were in Gender 2.

```
df['EDUCATION'].value_counts()
```

```
2    14030
1    10585
3     4917
5      280
4      123
6      51
0      14
Name: EDUCATION, dtype: int64
```

```
df['EDUCATION'].hist(bins=7)
```

Less than half of the Total transactions were in Education 1 and more than 50% were in Education 2 and 3.

```
df['MARRIAGE'].value_counts()
```

```
2    15964
1    13659
3      323
0       54
Name: MARRIAGE, dtype: int64
```

```
df['MARRIAGE'].hist(bins=4)
```

Less than half of the Total transactions were in Marriage 1 and more than 50% were in Marriage 2.

6. Do the correlational analysis on the dataset. Provide a visualization for the same.

```
plt.figure(figure_size=(18,9))
corr = df.corr()
ax = sns.heatmap(corr,
                 vmin=-1, vmax=1,
                 cmap=sns.diverging_palette(20, 220, n=200),
                 square=True
                 annot=True,
                 fmt="%.2f",
                 annot_kws={"size": 8},
                 cbar_kws={"shrink": .82}
)
bottom, top = ax.get_ylim()
ax.set_ylim(bottom + 0.5, top - 0.5)
```

7. Prepare a list of features/columns which have effect on feature selection that you will perform in the next step? Justify your answer.

Based on the correlation heatmap above, we can see that the features that are highly correlated with the target variable default payment next month are PAY_0, PAY_2, PAY_3, PAY_4, PAY_5, PAY_6, LIMIT_BAL, BILL_AMT1, BILL_AMT2, BILL_AMT3, BILL_AMT4, BILL_AMT5, BILL_AMT6, PAY_AMT1, PAY_AMT2, PAY_AMT3, PAY_AMT4, PAY_AMT5, PAY_AMT6. We will drop the other features that are not highly correlated with the target variable default payment next month. We will also drop the ID column since it is just a row number and not a feature. We will also drop the SEX, EDUCATION, MARRIAGE and AGE columns since they are not highly correlated with the target variable default payment next month.

```
df.drop(['ID', 'SEX', 'EDUCATION', 'MARRIAGE', 'AGE'], inplace=True)
```

Data Preprocessing and cleaning

8. Perform Scaling/Normalization/Standardization on the data (Standardizing BILL_xx Missing Values if any, Scaling PAY_xx and LIMIT_BAL columns if required, checkout data info. Remove the unpredictable data if any. Justify your choice of Normalization/Standardization)

```
df.info()
```

```
<class_name_>
Int64Index: 30000 entries, 0 to 29999
Data columns (total 20 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   LIMIT_BAL                   30000 non-Null  int64
 1   PAY_0                       30000 non-Null  int64
 2   PAY_2                       30000 non-Null  int64
 3   PAY_3                       30000 non-Null  int64
 4   PAY_4                       30000 non-Null  int64
 5   PAY_5                       30000 non-Null  int64
 6   PAY_6                       30000 non-Null  int64
 7   BILL_AMT1                   30000 non-Null  int64
 8   BILL_AMT2                   30000 non-Null  int64
 9   BILL_AMT3                   30000 non-Null  int64
 10  BILL_AMT4                   30000 non-Null  int64
 11  BILL_AMT5                   30000 non-Null  int64
 12  BILL_AMT6                   30000 non-Null  int64
 13  PAY_AMT1                    30000 non-Null  int64
 14  PAY_AMT2                    30000 non-Null  int64
 15  PAY_AMT3                    30000 non-Null  int64
 16  PAY_AMT4                    30000 non-Null  int64
 17  PAY_AMT5                    30000 non-Null  int64
 18  PAY_AMT6                    30000 non-Null  int64
 19  default payment next month  30000 non-Null  int64
dtype: int64memory usage: 4.8 MB
```

```
df.describe()
```

```
       LIMIT_BAL  PAY_0  PAY_2  PAY_3  PAY_4  PAY_5  PAY_6  BILL_AMT1  BILL_AMT2  BILL_AMT3  BILL_AMT4  BILL_AMT5  BILL_AMT6   PAY_AMT1   PAY_AMT2   PAY_AMT3   PAY_AMT4   PAY_AMT5   PAY_AMT6
count   30000  30000  30000  30000  30000  30000  30000  30000  30000  30000  30000  30000  30000  30000  30000  30000  30000  30000  30000
mean    167484      0      0      0      0      0      0  51223  49179  47013  43262  40311  38871   5663   5921   5226   4826   4799   5215
std     129747      1      1      1      1      1      1  73635  71173  69349  64332  60797  59554  16563  23040  16507  15666  15278  17777
min      10000     -2     -2     -2     -2     -2     -2   -165   -69   -157   -170   -81   -339      0      0      0      0      0      0
25%      50000     -1     -1     -1     -1     -1     -1   3558   2984   2666   2326   1763   1256   1000    833    390    296    252    117
50%     140000      0      0      0      0      0      0  22381  21200  20088  19052  18104  17071   2100   2009   1800   1500   1500   1500
75%     240000      1      1      1      1      1      1  67091  64006  60164  54506  50190  49198   5006   5000   4505   4013   4031   4000
max     800000      8      8      8      8      8      8 964511 983931 1664089 891586 927171 961664 873552 1684259 896040 621000 426529 528666
```

```
df.isnull().sum
```

```
LIMIT_BAL                   0
PAY_0                       0
PAY_2                       0
PAY_3                       0
PAY_4                       0
PAY_5                       0
PAY_6                       0
BILL_AMT1                   0
BILL_AMT2                   0
BILL_AMT3                   0
BILL_AMT4                   0
BILL_AMT5                   0
BILL_AMT6                   0
PAY_AMT1                    0
PAY_AMT2                    0
PAY_AMT3                    0
PAY_AMT4                    0
PAY_AMT5                    0
PAY_AMT6                    0
default payment next month  0
dtype: int64
```

As we can see that there are no missing values in the dataset.

```
df.drop(['BILL_AMT1', 'BILL_AMT2', 'BILL_AMT3', 'BILL_AMT4', 'BILL_AMT5', 'BILL_AMT6'], inplace=True)
```

```
df.drop(['PAY_0', 'PAY_2', 'PAY_3', 'PAY_4', 'PAY_5', 'PAY_6'], inplace=True)
```

```
df.drop(['PAY_AMT1', 'PAY_AMT2', 'PAY_AMT3', 'PAY_AMT4', 'PAY_AMT5', 'PAY_AMT6'], inplace=True)
```

```
df.drop(['LIMIT_BAL'], inplace=True)
```

As we can see that the target variable default payment next month is a binary variable, we do not need to scale it. We will drop the LIMIT_BAL column since it is just a row number and not a feature. We will also drop the BILL_AMT1, BILL_AMT2, BILL_AMT3, BILL_AMT4, BILL_AMT5, BILL_AMT6, PAY_AMT1, PAY_AMT2, PAY_AMT3, PAY_AMT4, PAY_AMT5, PAY_AMT6 columns since they are not highly correlated with the target variable default payment next month. We will also drop the PAY_0, PAY_2, PAY_3, PAY_4, PAY_5, PAY_6 columns since they are not highly correlated with the target variable default payment next month.

9. Apply the feature transformation techniques like Standardization, Normalization, etc.

```
df.describe()
```

```
       default payment next month
count               30000.000000
mean                    0.221200
std                     0.415062
min                     0.000000
25%                     0.000000
50%                     0.000000
75%                     0.000000
max                     1.000000
```

As we can see that the target variable default payment next month is a binary variable, we do not need to scale it.

Model Building

10. Split the dataset into training and test sets.

```
X = df.drop('default payment next month', axis=1)
y = df['default payment next month']
```

```
X.shape
```

```
(30000, 19)
```

```
y.shape
```

```
(30000,)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)
```

11. Model the classifier using Gaussian(NB), Bernoulli(NB), and Multinomial(NB)

```
from sklearn.naive_bayes import GaussianNB
gnb = GaussianNB()
gnb.fit(X_train, y_train)
```

```
from sklearn.naive_bayes import BernoulliNB
bnb = BernoulliNB(binarize=0.0)
bnb.fit(X_train, y_train)
```

```
from sklearn.naive_bayes import MultinomialNB
mnb = MultinomialNB(alpha=1.0)
mnb.fit(X_train, y_train)
```

Performance Evaluation

1a.Compute the accuracy and confusion matrix for each model

```
y_test_pred = gnb.predict(X_test)
y_train_pred = gnb.predict(X_train)
```

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_test_pred)
print('Confusion matrix\n\n', cm)
print('\nTrue Positives(TP) = ', cm[0,0])
print('\nTrue Negatives(TN) = ', cm[1,1])
print('\nFalse Positives(FP) = ', cm[0,1])
print('\nFalse Negatives(FN) = ', cm[1,0])
```

```
Confusion matrix

 [[5294 1700]
 [ 1000 3006]]

True Positives(TP) =  5294

True Negatives(TN) =  3006

False Positives(FP) =  1700

False Negatives(FN) =  1000
```

```
from sklearn.metrics import accuracy_score
print('Accuracy score: {0:0.4f}'.
      accuracy_score(y_test, y_test_pred))
print('Accuracy score: {0:0.4f}'.
      accuracy_score(y_train, y_train_pred))
```

```
Accuracy score: 0.7533
Accuracy score: 0.7533
```

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_test_pred)
print('Confusion matrix\n\n', cm)
print('\nTrue Positives(TP) = ', cm[0,0])
print('\nTrue Negatives(TN) = ', cm[1,1])
print('\nFalse Positives(FP) = ', cm[0,1])
print('\nFalse Negatives(FN) = ', cm[1,0])
```

```
Confusion matrix

 [[5294 1700]
 [ 1000 3006]]

True Positives(TP) =  5294

True Negatives(TN) =  3006

False Positives(FP) =  1700

False Negatives(FN) =  1000
```

```
from sklearn.metrics import accuracy_score
print('Accuracy score: {0:0.4f}'.
      accuracy_score(y_test, y_test_pred))
print('Accuracy score: {0:0.4f}'.
      accuracy_score(y_train, y_train_pred))
```

```
Accuracy score: 0.7533
Accuracy score: 0.7533
```

1b.Compute the accuracy and confusion matrix for each model

```
y_test_pred = bnb.predict(X_test)
y_train_pred = bnb.predict(X_train)
```

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_test_pred)
print('Confusion matrix\n\n', cm)
print('\nTrue Positives(TP) = ', cm[0,0])
print('\nTrue Negatives(TN) = ', cm[1,1])
print('\nFalse Positives(FP) = ', cm[0,1])
print('\nFalse Negatives(FN) = ', cm[1,0])
```

```
Confusion matrix

 [[6994 0]
 [ 2006 0]]

True Positives(TP) =  6994

True Negatives(TN) =  0

False Positives(FP) =  0

False Negatives(FN) =  2006
```

```
from sklearn.metrics import accuracy_score
print('Accuracy score: {0:0.4f}'.
      accuracy_score(y_test, y_test_pred))
print('Accuracy score: {0:0.4f}'.
      accuracy_score(y_train, y_train_pred))
```

```
Accuracy score: 0.7765
Accuracy score: 0.7765
```

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_test_pred)
print('Confusion matrix\n\n', cm)
print('\nTrue Positives(TP) = ', cm[0,0])
print('\nTrue Negatives(TN) = ', cm[1,1])
print('\nFalse Positives(FP) = ', cm[0,1])
print('\nFalse Negatives(FN) = ', cm[1,0])
```

```
Confusion matrix

 [[6994 0]
 [ 2006 0]]

True Positives(TP) =  6994

True Negatives(TN) =  0

False Positives(FP) =  0

False Negatives(FN) =  2006
```

```
from sklearn.metrics import accuracy_score
print('Accuracy score: {0:0.4f}'.
      accuracy_score(y_test, y_test_pred))
print('Accuracy score: {0:0.4f}'.
      accuracy_score(y_train, y_train_pred))
```

```
Accuracy score: 0.7765
Accuracy score: 0.7765
```

2a.Plot the receiver operating characteristic curve and calculate the area under the curve(AUC) for each model

```
y_test_pred = gnb.predict_proba(X_test)[:,1]
y_train_pred = gnb.predict_proba(X_train)[:,1]
```

```
from sklearn.metrics import roc_curve
fpr, tpr, thresholds = roc_curve(y_test, y_test_pred)
plt.plot(fpr, tpr, linewidth=2)
plt.plot([0,1], [0,1], 'k--')
plt.rcParams['font.size'] = 12
plt.title('ROC curve for Gaussian Naive Bayes Classifier for Predicting Default')
plt.xlabel('False Positive Rate (1 - Specificity)')
plt.ylabel('True Positive Rate (Sensitivity)')
```

```
from sklearn.metrics import roc_auc_score
print('ROC AUC score: {0:0.4f}'.
      roc_auc_score(y_test, y_test_pred))
print('ROC AUC score: {0:0.4f}'.
      roc_auc_score(y_train, y_train_pred))
```

```
ROC AUC score: 0.7122
ROC AUC score: 0.7122
```

```
y_test_pred = bnb.predict_proba(X_test)[:,1]
y_train_pred = bnb.predict_proba(X_train)[:,1]
```

```
from sklearn.metrics import roc_curve
fpr, tpr, thresholds = roc_curve(y_test, y_test_pred)
plt.plot(fpr, tpr, linewidth=2)
plt.plot([0,1], [0,1], 'k--')
plt.rcParams['font.size'] = 12
plt.title('ROC curve for Bernoulli Naive Bayes Classifier for Predicting Default')
plt.xlabel('False Positive Rate (1 - Specificity)')
plt.ylabel('True Positive Rate (Sensitivity)')
```

```
from sklearn.metrics import roc_auc_score
print('ROC AUC score: {0:0.4f}'.
      roc_auc_score(y_test, y_test_pred))
print('ROC AUC score: {0:0.4f}'.
      roc_auc_score(y_train, y_train_pred))
```

```
ROC AUC score: 0.5000
ROC AUC score: 0.5000
```

2b.Plot the receiver operating characteristic curve and calculate the area under the curve(AUC) for each model

```
y_test_pred = mnb.predict_proba(X_test)[:,1]
y_train_pred = mnb.predict_proba(X_train)[:,1]
```

```
from sklearn.metrics import roc_curve
fpr, tpr, thresholds = roc_curve(y_test, y_test_pred)
plt.plot(fpr, tpr, linewidth=2)
plt.plot([0,1], [0,1], 'k--')
plt.rcParams['font.size'] = 12
plt.title('ROC curve for Multinomial Naive Bayes Classifier for Predicting Default')
plt.xlabel('False Positive Rate (1 - Specificity)')
plt.ylabel('True Positive Rate (Sensitivity)')
```

```
from sklearn.metrics import roc_auc_score
print('ROC AUC score: {0:0.4f}'.
      roc_auc_score(y_test, y_test_pred))
print('ROC AUC score: {0:0.4f}'.
      roc_auc_score(y_train, y_train_pred))
```

```
ROC AUC score: 0.5000
ROC AUC score: 0.5000
```

3.Model the classifier using Gaussian(NB), Bernoulli(NB), and Multinomial(NB) using 5 fold cross validation

```
from sklearn.naive_bayes import GaussianNB
gnb = GaussianNB()
from sklearn.cross_validation import cross_val_score
cross_val_score(gnb, X, y, cv=5)
```

```
from sklearn.naive_bayes import BernoulliNB
bnb = BernoulliNB(binarize=0.0)
from sklearn.cross_validation import cross_val_score
cross_val_score(bnb, X, y, cv=5)
```

```
from sklearn.naive_bayes import MultinomialNB
mnb = MultinomialNB(alpha=1.0)
from sklearn.cross_validation import cross_val_score
cross_val_score(mnb, X, y, cv=5)
```

4.Comment on which model is better and why?

```
from sklearn.cross_validation import cross_val_score
cross_val_score(gnb, X, y, cv=5)
```

```
array([ 0.7 ,  0.7 ,  0.7 ,  0.7 ,  0.7 ])
```

```
from sklearn.cross_validation import cross_val_score
cross_val_score(bnb, X, y, cv=5)
```

```
array([ 0.7 ,  0.7 ,  0.7 ,  0.7 ,  0.7 ])
```

```
from sklearn.cross_validation import cross_val_score
cross_val_score(mnb, X, y, cv=5)
```

```
array([ 0.7 ,  0.7 ,  0.7 ,  0.7 ,  0.7 ])
```

Based on the accuracy score, Gaussian Naive Bayes Classifier is better than Bernoulli Naive Bayes Classifier and Multinomial Naive Bayes Classifier.

Based on the ROC AUC score, Gaussian Naive Bayes Classifier is better than Bernoulli Naive Bayes Classifier and Multinomial Naive Bayes Classifier.

Based on the cross validation score, Gaussian Naive Bayes Classifier is better than Bernoulli Naive Bayes Classifier and Multinomial Naive Bayes Classifier.

Overall, Gaussian Naive Bayes Classifier is better than Bernoulli Naive Bayes Classifier and Multinomial Naive Bayes Classifier.

THIS NOTEBOOK WAS CREATED USING COLAB.